

Student API — A to Z Exam Guidance Handbook

Spring Boot · Postman · Git · Docker · Jenkins

*Print this before the practical exam. Every code block and command below matches the **working** project.*

Item	Value
Local folder	C:\Users\rasar\OneDrive\Desktop\SOC
GitHub	https://github.com/Razaara/student-api
Docker Hub	razzara/student-api
Port	8500
Base URL	http://localhost:8500/api/students
DB name	students
DB user / pass	root / root
Java	21
Spring Boot	3.4.5

Table of Contents

1. [Exam workflow \(do this order\)](#)
 2. [Project structure](#)
 3. [Create Spring Boot project](#)
 4. [pom.xml](#)
 5. [application.properties](#)
 6. [Entity — Student.java](#)
 7. [Repository](#)
 8. [Service layer](#)
 9. [Controller](#)
 10. [Main class](#)
 11. [Run locally + MySQL](#)
 12. [Postman testing](#)
 13. [Git + GitHub](#)
 14. [Dockerfile](#)
 15. [docker-compose.yml](#)
 16. [Docker Hub push](#)
 17. [Jenkins pipeline](#)
 18. [Annotations cheat sheet](#)
 19. [Common errors and fixes](#)
 20. [Common Git issues](#)
 21. [Ports already in use — how to fix](#)
 22. [One-page final checklist](#)
-

1. Exam workflow (do this order)

1. Create Spring Boot project (Web, JPA, MySQL, Lombok, Validation)
2. Write application.properties (port 8500)
3. Create Entity → Repository → Service → Controller
4. Start MySQL
5. Run: mvn spring-boot:run
6. Test 5 endpoints in Postman
7. git add / commit / push to GitHub
8. docker build + docker run (or docker compose up)
9. Push image to Docker Hub
10. Create Jenkins Pipeline job → Build Now
11. Re-test API in Postman

If time is short: CRUD + Postman first → Git push → Docker → Jenkins last.

2. Project structure

```
SOC/
├─ pom.xml
├─ Dockerfile
├─ docker-compose.yml
├─ Jenkinsfile
├─ .gitignore
├─ README.md
└─ src/main/
    ├─ java/com/example/studentapi/
    │   ├─ StudentApiApplication.java
    │   ├─ entity/Student.java
    │   ├─ repository/StudentRepository.java
    │   ├─ service/StudentService.java
    │   └─ service/StudentServiceImpl.java
    └─ controller/StudentController.java
    └─ resources/application.properties
```

3. Create Spring Boot project

Option A — Spring Initializr (IDE)

1. <https://start.spring.io> or IntelliJ → New → Spring Initializr
2. Project: **Maven** · Language: **Java** · Java: **21**
3. Group: `com.example` · Artifact: `studentapi` · Package: `com.example.studentapi`
4. Dependencies:
 - Spring Web
 - Spring Data JPA
 - MySQL Driver
 - Lombok
 - Validation

5. Generate → extract into Desktop\SOC

Note: If Initializr only offers Spring Boot 4.x, choose Boot **3.4.x** if available, or create `pom.xml` manually with parent version `3.4.5` (this project).

Option B — Use this ready repo

```
cd C:\Users\rasar\OneDrive\Desktop
git clone https://github.com/Razaara/student-api.git SOC
cd SOC
```

4. pom.xml

Full working `pom.xml` :

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.4.5</version>
    <relativePath/>
  </parent>

  <groupId>com.example</groupId>
  <artifactId>studentapi</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>studentapi</name>
  <description>Student CRUD REST API for SOC practical exam</description>

  <properties>
    <java.version>21</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>com.mysql</groupId>
```

```

        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
                <excludes>
                    <exclude>
                        <groupId>org.projectlombok</groupId>
                        <artifactId>lombok</artifactId>
                    </exclude>
                </excludes>
            </configuration>
        </plugin>
    </plugins>
</build>
</project>

```

Spring Boot 3 uses `com.mysql:mysql-connector-j` (not old `mysql:mysql-connector-java`).

5. application.properties

File: `src/main/resources/application.properties`

```

spring.application.name=studentapi
spring.datasource.url=jdbc:mysql://localhost:3306/students?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
server.port=8500

```

Property	Why
createDatabaseIfNotExist=true	Creates DB if missing
ddl-auto=update	Auto-creates/updates tables from entity
server.port=8500	App listens on 8500 (not 8080)
password root	Matches Docker MySQL MYSQL_ROOT_PASSWORD

6. Entity — Student.java

File: `src/main/java/com/example/studentapi/entity/Student.java`

```
package com.example.studentapi.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Entity
@Table(name = "student")
@AllArgsConstructor
@NoArgsConstructor
public class Student {

    @Id
    private String id;
    private String name;
    private String email;
    private Integer age;
}
```

Must use `jakarta.persistence` in Spring Boot 3 (not `javax.persistence`).
`@Data` already includes getters/setters — no need for extra `@Getter` / `@Setter` .
`id` is a `String` (e.g. `AS2023000`) — send it in the `POST` body.

7. Repository

File: `src/main/java/com/example/studentapi/repository/StudentRepository.java`

```
package com.example.studentapi.repository;

import com.example.studentapi.entity.Student;
import org.springframework.data.jpa.repository.JpaRepository;
```

```
public interface StudentRepository extends JpaRepository<Student, String> {  
}
```

`JpaRepository<Student, String>` → entity type + primary key type.

8. Service layer

StudentService.java (interface)

```
package com.example.studentapi.service;  
  
import com.example.studentapi.entity.Student;  
import java.util.List;  
  
public interface StudentService {  
    Student save(Student student);  
    List<Student> getAll();  
    Student getById(String id);  
    Student update(String id, Student student);  
    void delete(String id);  
}
```

StudentServiceImpl.java

```
package com.example.studentapi.service;  
  
import com.example.studentapi.entity.Student;  
import com.example.studentapi.repository.StudentRepository;  
import lombok.RequiredArgsConstructor;  
import org.springframework.stereotype.Service;  
import java.util.List;  
  
@Service  
@RequiredArgsConstructor  
public class StudentServiceImpl implements StudentService {  
  
    private final StudentRepository studentRepository;  
  
    @Override  
    public Student save(Student student) {  
        return studentRepository.save(student);  
    }  
  
    @Override  
    public List<Student> getAll() {  
        return studentRepository.findAll();  
    }  
  
    @Override
```

```

    public Student getById(String id) {
        return studentRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Student not found with id: " +
id));
    }

    @Override
    public Student update(String id, Student student) {
        Student existing = getById(id);
        existing.setName(student.getName());
        existing.setEmail(student.getEmail());
        existing.setAge(student.getAge());
        return studentRepository.save(existing);
    }

    @Override
    public void delete(String id) {
        studentRepository.deleteById(id);
    }
}

```

Missing id throws `RuntimeException` → HTTP **500** unless you add exception handling.

9. Controller

File: `src/main/java/com/example/studentapi/controller/StudentController.java`

```

package com.example.studentapi.controller;

import com.example.studentapi.entity.Student;
import com.example.studentapi.service.StudentService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/students")
@RequiredArgsConstructor
public class StudentController {

    private final StudentService studentService;

    @PostMapping
    public ResponseEntity<Student> create(@RequestBody Student student) {
        Student saved = studentService.save(student);
        return new ResponseEntity<>(saved, HttpStatus.CREATED);
    }

    @GetMapping

```

```

    public ResponseEntity<List<Student>> getAll() {
        return ResponseEntity.ok(studentService.getAll());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Student> getById(@PathVariable String id) {
        return ResponseEntity.ok(studentService.getById(id));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Student> update(@PathVariable String id, @RequestBody Student
student) {
        return ResponseEntity.ok(studentService.update(id, student));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable String id) {
        studentService.delete(id);
        return ResponseEntity.noContent().build();
    }
}

```

Endpoint summary

Method	Path	Success
POST	/api/students	201 Created
GET	/api/students	200 OK
GET	/api/students/{id}	200 OK
PUT	/api/students/{id}	200 OK
DELETE	/api/students/{id}	204 No Content

10. Main class

```

package com.example.studentapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class StudentApiApplication {

    public static void main(String[] args) {
        SpringApplication.run(StudentApiApplication.class, args);
    }
}

```


Package of controllers/services must be **under** `com.example.studentapi` for component scan.

11. Run locally + MySQL

Start MySQL with Docker (recommended)

```
cd C:\Users\rasar\OneDrive\Desktop\SOC
docker compose up -d mysql
docker ps
```

Wait until `student-mysql` is **healthy**.

Or use local MySQL

```
CREATE DATABASE students;
-- user root / password root (match application.properties)
```

Run the app

```
mvn clean package -DskipTests
mvn spring-boot:run
```

Console must show:

```
Tomcat started on port(s): 8500
Started StudentApiApplication
```

12. Postman testing

Base URL: `http://localhost:8500/api/students`

Body type: **raw** → **JSON**

12.1 POST — Create

POST `http://localhost:8500/api/students`

```
{
  "id": "AS2023000",
  "name": "John Doe",
  "email": "john@example.com",
  "age": 21
}
```

Expect: **201** + same JSON back.

12.2 GET — All

GET http://localhost:8500/api/students

Expect: **200** + JSON array.

12.3 GET — By id

GET http://localhost:8500/api/students/AS2023000

Expect: **200** + one student.

12.4 PUT — Update

PUT http://localhost:8500/api/students/AS2023000

```
{
  "name": "John Updated",
  "email": "john.updated@example.com",
  "age": 22
}
```

Expect: **200**, "id": "AS2023000" unchanged.

12.5 DELETE

DELETE http://localhost:8500/api/students/AS2023000

Expect: **204** (empty body).

HTTP codes

Code	Meaning
200	OK (GET/PUT)
201	Created (POST)
204	No Content (DELETE)
400	Bad JSON / bad request
404	Wrong URL / not mapped
500	Server error (e.g. missing id → RuntimeException)

13. Git + GitHub

Common Git errors (auth, rejected push, wrong remote, shared PC): see [§20 Common Git issues](#).

.gitignore

```
target/
.idea/
*.iml
*.class
```

```
.env  
*.log
```

First push

```
cd C:\Users\rasar\OneDrive\Desktop\SOC  
git init  
git add .  
git commit -m "Initial commit: Student CRUD API"  
git branch -M main  
git remote add origin https://github.com/Razaara/student-api.git  
git push -u origin main
```

Everyday

```
git status  
git add .  
git commit -m "Add update endpoint"  
git push
```

Shared lab PC — clear old GitHub login

1. Windows → **Credential Manager** → **Windows Credentials**
2. Remove `github.com` / `git:https://github.com`
3. Then:

```
git config user.name "Your Name"  
git config user.email "your-email@example.com"  
git remote -v  
git push -u origin main
```

Login with **your** GitHub account when prompted.
(`user.name` / `user.email` do **not** log you into GitHub.)

This project's repo

<https://github.com/Razaara/student-api>

14. Dockerfile

File: `Dockerfile` (project root)

```
# --- Stage 1: Build the application with Maven ---  
FROM maven:3.9-eclipse-temurin-21 AS build  
WORKDIR /app  
COPY pom.xml .  
COPY src ./src  
RUN mvn -B clean package -DskipTests
```

```
# --- Stage 2: Run the packaged jar on a lightweight JRE ---
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8500
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Build & run (API only — MySQL must already be running)

```
docker build -t student-api .
docker run -d -p 8500:8500 --name student-api-container ^
  -e SPRING_DATASOURCE_URL=jdbc:mysql://host.docker.internal:3306/students?
createDatabaseIfNotExist=true ^
  -e SPRING_DATASOURCE_USERNAME=root ^
  -e SPRING_DATASOURCE_PASSWORD=root ^
  student-api

docker logs -f student-api-container
docker stop student-api-container
docker rm student-api-container
```

Inside a container, `localhost` is **not** your PC. Use `host.docker.internal` (Docker Desktop) or the Compose service name `mysql`.

Useful Docker commands

```
docker images
docker ps
docker ps -a
docker stop <id>
docker rm <id>
docker rmi <image>
docker logs <id>
docker login
docker tag student-api razzara/student-api
docker push razzara/student-api
```

15. docker-compose.yml

File: `docker-compose.yml`

```
services:
  mysql:
    image: mysql:8.0
    container_name: student-mysql
    restart: always
    environment:
```

```

    MYSQL_ROOT_PASSWORD: root
    MYSQL_DATABASE: students
ports:
  - "3306:3306"
volumes:
  - mysql_data:/var/lib/mysql
healthcheck:
  test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-proot"]
  interval: 10s
  timeout: 5s
  retries: 10
  start_period: 20s

springboot:
  build: .
  container_name: student-api
  restart: on-failure
  ports:
    - "8500:8500"
  environment:
    SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/students?createDatabaseIfNotExist=true
    SPRING_DATASOURCE_USERNAME: root
    SPRING_DATASOURCE_PASSWORD: root
  depends_on:
    mysql:
      condition: service_healthy

volumes:
  mysql_data:

```

Compose commands

```

docker compose up -d --build
docker compose ps
docker compose logs -f springboot
docker compose down

```

In Compose, DB host is service name `mysql`, not `localhost`.

16. Docker Hub push

Docker username: `razzara`

```

docker login
docker tag student-api:latest razzara/student-api:latest
docker push razzara/student-api:latest

```

Pull later:

```
docker pull razzara/student-api:latest
```

17. Jenkins pipeline

17.1 Jenkins tools (one-time)

Manage Jenkins → Tools

- Maven installation **Name:** Maven (must match Jenkinsfile)
- JDK: system Java 21 is fine on this machine

Docker must be available to the Jenkins agent (Docker Desktop running).

17.2 Create Pipeline job

1. Open <http://localhost:8080>
2. Login
3. **New Item** → name: student-api → **Pipeline** → OK
4. Pipeline → **Pipeline script from SCM**
5. SCM: **Git**
6. Repository URL: <https://github.com/Razaara/student-api.git>
7. Branch: */main
8. Script Path: Jenkinsfile
9. Save → **Build Now**

17.3 Working Jenkinsfile

```
pipeline {
    agent any

    tools {
        // Must match Manage Jenkins → Tools → Maven installations → Name
        maven 'Maven'
    }

    environment {
        IMAGE_NAME = 'student-api'
        CONTAINER_NAME = 'student-api-container'
        // Host MySQL from docker-compose service student-mysql on port 3306
        DB_URL = 'jdbc:mysql://host.docker.internal:3306/students?createDatabaseIfNotExist=true'
        DB_USER = 'root'
        DB_PASS = 'root'
    }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/Razaara/student-api.git'
            }
        }
    }
}
```

```

stage('Build with Maven') {
    steps {
        bat 'mvn clean package -DskipTests'
    }
}

stage('Build Docker Image') {
    steps {
        bat 'docker build -t %IMAGE_NAME% .'
    }
}

stage('Stop Old Container') {
    steps {
        bat 'docker stop %CONTAINER_NAME% 2>NUL || exit /b 0'
    }
}

stage('Remove Old Container') {
    steps {
        bat 'docker rm %CONTAINER_NAME% 2>NUL || exit /b 0'
    }
}

stage('Run New Container') {
    steps {
        bat '''
            docker run -d ^
            -p 8500:8500 ^
            --name %CONTAINER_NAME% ^
            -e SPRING_DATASOURCE_URL=%DB_URL% ^
            -e SPRING_DATASOURCE_USERNAME=%DB_USER% ^
            -e SPRING_DATASOURCE_PASSWORD=%DB_PASS% ^
            %IMAGE_NAME%
            ...
        '''
    }
}

stage('Verify') {
    steps {
        bat '''
            setlocal EnableDelayedExpansion
            set OK=0
            for /L %i in (1,1,18) do (
                curl.exe -f -s -o NUL http://localhost:8500/api/students
                if !ERRORLEVEL! EQU 0 (
                    echo API is UP
                    set OK=1
                    goto :done
                )
            )
            echo Waiting for API... attempt %i
        '''
    }
}

```

```

        ping -n 6 127.0.0.1 >NUL
    )
    :done
    if !OK! NEQ 1 (
        echo API did not become ready
        docker logs %CONTAINER_NAME%
        exit /b 1
    )
    curl.exe -s http://localhost:8500/api/students
    echo.
    ...
}
}
}

post {
    success {
        echo 'Deployment successful. API: http://localhost:8500/api/students'
    }
    failure {
        echo 'Pipeline failed – check console output.'
        bat 'docker logs %CONTAINER_NAME% 2>NUL || exit /b 0'
    }
}
}
}

```

Pipeline stages (what examiner sees)

Checkout → Build with Maven → Build Docker Image →
 Stop Old Container → Remove Old Container → Run New Container → Verify

Before Build Now

```

# MySQL must be up; free port 8500 if something else uses it
docker compose up -d mysql
docker stop student-api 2>NUL
docker stop student-api-container 2>NUL

```

Verified: Jenkins job `student-api` build **#1 SUCCESS**.

18. Annotations cheat sheet

Annotation	Use
<code>@SpringBootApplication</code>	Main class — auto-config + component scan
<code>@RestController</code>	REST controller (<code>@Controller</code> + <code>@ResponseBody</code>)
<code>@RequestMapping</code>	Base path, e.g. <code>/api/students</code>

@PostMapping / @GetMapping / @PutMapping / @DeleteMapping	HTTP methods
@PathVariable	Bind {id} from URL
@RequestBody	Bind JSON body to Java object
@Service	Service bean
@Entity / @Id / @Table	JPA entity
@Data	Lombok getters/setters/toString>equals
@RequiredArgsConstructor	Constructor injection for final fields

19. Common errors and fixes

Error	Fix
Port 8500 / 3306 / 8080 already in use	See §21
Communications link failure	Start MySQL: <code>docker compose up -d mysql</code>
Access denied for user root	Password must be root (match Compose)
Unknown database	Use <code>createDatabaseIfNotExist=true</code>
Whitelabel 404	Check URL path + <code>@RequestMapping</code>
<code>javax.persistence</code> not found	Change to <code>jakarta.persistence</code>
Lombok getters missing in IDE	Install Lombok plugin + enable annotation processing
Docker push denied	<code>docker login</code> + tag as <code>razzara/student-api</code>
Jenkins <code>mvn</code> not found	Tools name must be exactly Maven
Jenkins container can't reach DB	Use <code>host.docker.internal</code> (as in Jenkinsfile)
Git problems	See §20
POST 400 in PowerShell curl	JSON escaping broken — use Postman instead

20. Common Git issues

Use these during the exam when `git push` / `commit` fails.

Issue / Error	Cause	Fix
<code>fatal: not a git repository</code>	Wrong folder or never ran <code>git init</code>	<code>cd</code> into project folder, then <code>git init</code>
<code>fatal: remote origin already exists</code>	You already added origin	<code>git remote set-url origin https://github.com/Razaara/student-api.git</code>

Updates were rejected / non-fast-forward	Remote has commits you don't have	<code>git pull origin main</code> (Or <code>git pull --rebase</code>) then <code>git push</code>
Authentication failed / Password authentication is not supported	GitHub blocks account password for HTTPS	Use Personal Access Token (PAT) as password, or sign in via Git Credential Manager
Wrong GitHub account on shared PC	Old credentials stored in Windows	Credential Manager → Windows Credentials → remove <code>github.com</code> → push again and login
Permission denied (publickey)	SSH key not set up	Switch remote to HTTPS: <code>git remote set-url origin https://github.com/YOUR_USER/YOUR_REPO.git</code>
Everything up-to-date but files missing on GitHub	Forgot to commit	<code>git status</code> → <code>git add .</code> → <code>git commit -m "msg"</code> → <code>git push</code>
Accidentally committed <code>target/</code>	Build folder should be ignored	Add <code>target/</code> to <code>.gitignore</code> , then <code>git rm -r --cached target/</code> → <code>commit</code> → <code>push</code>
Merge conflict (<<<<<<)	Same lines changed in two commits	Edit file, remove markers, keep correct code → <code>git add .</code> → <code>git commit</code>
<code>fatal: refusing to merge unrelated histories</code>	New local repo + existing GitHub repo	<code>git pull origin main --allow-unrelated-histories</code> then <code>resolve / push</code>
Wrong commit message (not pushed yet)	Typo in last commit	<code>git commit --amend -m "correct message"</code>
Detached HEAD	Checked out a commit hash, not a branch	<code>git checkout main</code>
Please tell me who you are	Git identity not set	<code>git config user.name "Your Name"</code> and <code>git config user.email "you@email.com"</code>

Quick Git recovery commands

```
# See what is wrong
git status
git remote -v
git log --oneline -5

# Fix wrong remote URL
git remote set-url origin https://github.com/Razaara/student-api.git

# Pull then push (rejected updates)
git pull origin main
git push origin main

# Remove tracked target/ after adding .gitignore
```

```
git rm -r --cached target/
git add .
git commit -m "Stop tracking target folder"
git push

# Clear old GitHub credential (Windows)
# Search: Credential Manager → Windows Credentials → Remove github.com entries
# Then:
git push
# Login popup → use YOUR GitHub account / PAT
```

Shared university PC checklist

1. Credential Manager → remove old github.com credentials
2. `git config user.name "Your Name"`
3. `git config user.email "your-email@example.com"`
4. `git remote -v` (must be YOUR repo)
5. `git add .` → `git commit -m "..."` → `git push`
6. Login with YOUR GitHub account when prompted

`git config user.name` / `user.email` only set the **author name on commits**. They do **not** authenticate you.
Authentication = GitHub login / PAT / Credential Manager.

21. Ports already in use — how to fix

This project / tools commonly use:

Port	Used by
8500	Spring Boot Student API
3306	MySQL
8080	Jenkins

Symptoms

- Spring Boot: Port 8500 was already in use
- Docker: Bind for 0.0.0.0:8500 failed: port is already allocated
- Jenkins / browser: wrong app opens, or service won't start

Step 1 — See what is using the port (Windows)

PowerShell:

```
# Replace 8500 with 3306 or 8080 as needed
netstat -ano | findstr :8500
```

Last column = **PID** (process id).

Or:

```
Get-NetTCPConnection -LocalPort 8500 -ErrorAction SilentlyContinue |  
    Select-Object LocalPort,OwningProcess,State
```

Step 2A — Stop Docker containers (most common in this exam)

```
docker ps  
docker stop student-api  
docker stop student-api-container  
docker rm student-api  
docker rm student-api-container  
  
# If MySQL port 3306 is stuck on an old container:  
docker stop student-mysql  
docker rm student-mysql  
  
# Or stop everything from this project:  
cd C:\Users\rasar\OneDrive\Desktop\SOC  
docker compose down
```

Then start only what you need:

```
docker compose up -d mysql  
# later run app with mvn OR full compose / Jenkins
```

Step 2B — Kill a normal Windows process (Java / old Spring Boot)

```
# Find PID for port 8500  
netstat -ano | findstr :8500  
  
# Kill that PID (example: 12345)  
taskkill /PID 12345 /F
```

Or one-liner:

```
Get-NetTCPConnection -LocalPort 8500 -ErrorAction SilentlyContinue |  
    ForEach-Object { Stop-Process -Id $_.OwningProcess -Force -ErrorAction SilentlyContinue }
```

Repeat for 3306 or 8080 if needed.

Step 2C — Jenkins / Docker Desktop still holding the port

```
# Check containers again  
docker ps -a  
  
# Force remove container using the port  
docker rm -f student-api student-api-container 2>$null
```

```
# If Jenkins itself must restart (port 8080)
# Services → Jenkins → Restart (or)
Restart-Service Jenkins
```

Step 3 — Change the port (only if you cannot free it)

Spring Boot — in `application.properties` :

```
server.port=8501
```

Then Postman / Docker must use **8501** too:

```
docker run -d -p 8501:8501 --name student-api-container student-api
```

Docker host mapping (keep app on 8500 inside container, map different host port):

```
docker run -d -p 8501:8500 --name student-api-container student-api
```

Postman URL becomes: `http://localhost:8501/api/students`

*Prefer **freeing the port** over changing it mid-exam so handbook URLs stay correct.*

Fast exam fix for port 8500

```
docker ps
docker stop student-api student-api-container 2>NUL
docker rm student-api student-api-container 2>NUL
netstat -ano | findstr :8500
# if a PID remains:
taskkill /PID <PID> /F
mvn spring-boot:run
```

Fast exam fix for port 3306

```
docker ps
docker stop student-mysql 2>NUL
docker rm student-mysql 2>NUL
docker compose up -d mysql
```

Fast exam fix for port 8080 (Jenkins)

```
# See who uses 8080
netstat -ano | findstr :8080
# If another app stole 8080, stop it, then start Jenkins service
Get-Service Jenkins
Start-Service Jenkins
```

22. One-page final checklist

Commands you will type

```
# MySQL
docker compose up -d mysql

# Run API locally
mvn spring-boot:run

# Git
git add .
git commit -m "Student CRUD API"
git push -u origin main

# Docker
docker build -t student-api .
docker compose up -d --build
docker tag student-api razzara/student-api
docker push razzara/student-api

# Jenkins
# Browser → http://localhost:8080 → student-api → Build Now
```

Postman order

POST (201) → GET all (200) → GET by id (200) → PUT (200) → DELETE (204)

Remember

- Port **8500** everywhere
- Include **id** in POST body
- `jakarta.persistence` not `javax`
- Compose DB host = `mysql`
- Jenkins docker run DB host = `host.docker.internal`
- GitHub user: **Razaara** · Docker user: **razzara**

Viva quick answers

- `@Component` vs `@Service` vs `@Repository` — same stereotype, different layer meaning
- DI — Spring injects beans (prefer constructor / `@RequiredArgsConstructor`)
- `JpaRepository` vs `CrudRepository` — Jpa adds paging/sorting/JPA helpers
- Image vs Container — image = template; container = running instance
- Declarative Jenkins pipeline — `pipeline { stages { ... } }` in Jenkinsfile

End of handbook.

Project path: `C:\Users\rasar\OneDrive\Desktop\SOC`

Repo: <https://github.com/Razaara/student-api>